Computer Vision, Assignment 4
Model Fitting and Optimization

1 Instructions

The data for the assignments is available from the course page:

`https://canvas.education.lu.se/courses/39473`

Deadline:
Hand in on time via Canvas by study week 8, 15/3 at midnight. You must attempt all mandatory exercises (no blank answers). Extensions are only given in exceptional cases after emailing the lecturer in advance.

Report format:
Submit one PDF with complete solutions. Use LaTeX for typesetting the report. Handwritten solutions to some exercises are allowed if scanned into the PDF and clearly structured and readable.

Code submission:
Submit all code in a single zip file, including any data. Code must be well commented and directly runnable (no manual loading of data, setting parameters, etc.).

Help and collaboration:
You may work together and ask questions in Q&A sessions or by emailing the lecturer, but the report must be your own work. You may not use AI tools (e.g. ChatGPT or other LLMs) to directly construct your solutions. You must understand and be able to explain **everything** you submit, and you may be asked to clarify parts of your work if there is any doubt about authorship.

Corrections:
If any part of your submission is incorrect, you may submit corrections. You are allowed a maximum of two correction submissions.

Bonus:
The OPTIONAL exercises are not required to pass but give bonus points on the take-home exam if solved well. You can earn up to 0.25 points per assignment.

2 Robust Homography Estimation and Stitching

Exercise 1.

a) What are the camera centers for the cameras $P_1 = [A_1\ t_1]$ and $P_2 = [A_2\ t_2]$.
(You can assume that A_1 and A_2 are invertible.)

b) Show that if the two cameras P_1 and P_2 have the same camera center then there is a homography H that transforms the the first image into the second one.

For the report: Complete solutions for (***a***) and (***b***).

Exercise 2. Suppose that we want to find a homography that transforms one 2D point set into another.

a) How many degrees of freedom does a homography have?

b) What is the minimal number of point correspondences that you need to determine the homography?

c) If the number of incorrect correspondences is 45% how many iterations of RANSAC do you need to find an outlier free sample set with 99% probability?

For the report: Answers for (***a***), (***b***) and (***c***). For (***c***) make sure to also include the calculations showing how you arrived at your answer.

Exercise 3. In this exercise you will use RANSAC to estimate an image homography.

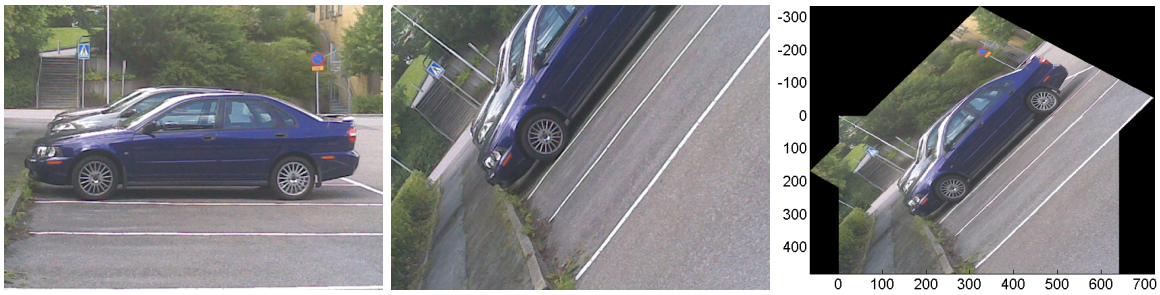

Figure 1: Image `A4_ex3_im1.jpg`, `A4_ex3_im2.jpg` and a panorama.

You can use the two images `A4_ex3_im1.jpg` and `A4_ex3_im2.jpg` (see Figure 1), but feel free to use other images if you want to. To run SIFT matching on your own images you can see the hints below. For the two given images the SIFT matches can be found in `A4_ex3_data.npz`.

a) Begin by loading the images and plotting the matches. The images are partly overlapping.

The goal is now to place them on top of each other as in Figure 1 by estimating a homography from the matched points. Because not all matches are correct, you need to use RANSAC to find a set of good correspondences (inliers).

b) Implement a function that estimates a homography from a minimal number of points. Note that in this case the least squares system will have an exact solution, so normalization is not necessary.

c) Implement a RANSAC for estimating a homography. A reasonable threshold for inliers is 5 pixels.

How many inliers did you find?

d) Finally, transform the images to a common coordinate system using the estimated homography. You can use the `utils.homography_warp_image` function (see hints section).

For the report: The code, a plot of the panorama (and the two images if you don't use a.jpg and b.jpg), and answers to all the questions.

```
Hints:

# Warp im1 to im2's coordinate system and get the bounds
im1_warp, bounds = utils.homography_warp_image(im1, H, bounds=None)
# Warp im2 using the same bounds (identity homography keeps it unchanged)
im2_warp, _ = utils.homography_warp_image(im2, np.eye(3), bounds=bounds)

# Average where both images overlap, otherwise use whichever image has content
im1_mask = np.any(im1_warp > 0, axis=-1)
im2_mask = np.any(im2_warp > 0, axis=-1)
blend = (im1_warp.astype(float) + im2_warp.astype(float)) / 2
overlay = im2_warp.copy()
overlay[im1_mask & ~im2_mask] = im1_warp[im1_mask & ~im2_mask]
overlay[im1_mask & im2_mask] = blend[im1_mask & im2_mask].astype(overlay.dtype)

# Optional:
# We can use the SIFT implementation in scikit-image
# (install with `pip install -U scikit-image`)
from skimage.feature import SIFT
sift = SIFT()
sift.detect_and_extract(img1)
keypoints1 = descriptor_extractor.keypoints
desc1 = descriptor_extractor.descriptors
sift.detect_and_extract(img2)
keypoints2 = descriptor_extractor.keypoints
desc2 = descriptor_extractor.descriptors
matches = match_descriptors(desc1, desc2, max_ratio=0.6, cross_check=True)
x1 = keypoints1[matches[:, 0]][:, ::-1].T
x2 = keypoints2[matches[:, 1]][:, ::-1].T
```

3 Robust Essential Matrix Estimation

Exercise 4. Suppose that we want to estimate the epipolar geometry between two calibrated cameras.

a) How many degrees of freedom does an Essential matrix have?

b) What is the minimal number of point correspondences that you need to determine the essential matrix?

c) If the number of incorrect correspondences is 45% how many iterations of RANSAC do you need to find an outlier free sample set with 99% probability?

For the report: Answers for (***a***), (***b***) and (***c***). For (***c***) make sure to also include the calculations showing how you arrived at your answer.

Figure 2: Two images of the Basilica di Santa Maria del Fiore, Florence.

Exercise 5. The file `A4_ex5_data.npz` contains image points (`x1` and `x2`) matched using SIFT descriptors from the images in Figure 2.

a) The function `utils.fivepoint_solver` solves for the essential matrix from 5 point correspondences. Using this function implement a RANSAC that estimates the essential matrix. Note that the solver gives multiple solutions and you have to test them all in the RANSAC loop.

For a match to be considered an inlier its distance to the corresponding epipolar line should be less than 5 pixels (in both images). To properly compute this distance you will need the camera calibration, which is the same for both images and can be found in the variable `K`.

b) How many inliers did you find?

c) Extract cameras from the best essential matrix and triangulate the 3D points from the inlier matches. Plot the 3D points together with the cameras.

Make sure that the points are in front of BOTH cameras. Note that you can use your code from Exercise 11b in Assignment 3 for this part.

d) Plot a histogram of the reprojection errors in both images (only using the inliers, with unnormalized camera matrices and errors measured in pixels) and compute the average reprojection error in each image.

For the report: Submit the code, the number of inliers (***b***), a plot of the reconstruction from (***c***), the histogram and reprojection errors from (***d***).

4 Calibrated Structure from Motion and Local Optimization

Exercise 6. Unfortunately, there is no closed-form solution for computing the maximum-likelihood (ML) estimate when using general pinhole camera models. Consequently, the ML estimate must be obtained by refining an initial solution through local optimization. In general, this requires minimizing an objective function of the form

$$\sum_i r_i(v)^2 = \|\mathbf{r}(v)\|^2, \tag{1}$$

where v denotes the set of optimizable variables: camera positions, camera orientations, and 3D point coordinates. Here $r_i(v)$ is the reprojection error associated with the i-th observation. The vector $\mathbf{r}(v)$ stacks all residuals $r_i(v)$ into a single residual vector.

The first order Taylor expansion at a point v_k is

$$\mathbf{r}(v) \approx \mathbf{r}(v_k) + J(v_k)\delta v, \tag{2}$$

where $\delta v = (v - v_k)$ and the Jacobian $J(v_k)$ is a matrix whose rows are the gradients of $r_i(v)$ at v_k.

In this exercise we will solve this optimization problem. To help we have the following helper functions:

- Given a current solution consisting of cameras and 3D-points the function

  `r, J = utils.linearize_reprojection_error(P, x, X)`

  computes the values of $\mathbf{r}$ and J.
- The function `Pnew, Xnew = utils.update_solution(P, X, delta)` contains a function that computes a new set of cameras and 3D points from an update δv computed by any method.
- The function `res = compute_reprojection_rms_error(P, x, X)` computes the Root-Mean-Squared (RMS) reprojection error (i.e. `res` $= \|\mathbf{r}\|/\sqrt{N}$, where N is the number of residuals) for a given set of cameras, 3D points and image points.

The steepest descent method updates the solution by searching along the direction opposite to the gradient $\nabla\|\mathbf{r}(v)\|^2 = 2J(v_k)^T\mathbf{r}(v_k)$ giving the update

$$\delta v = -\gamma_k J(v_k)^T r(v_k). \tag{3}$$

Here γ_k is some unknown positive number that is determined by testing different values and evaluating the objective function. Note that unless v_k is a local minimizer it is always possible to find γ_k so that the objective function decreases by making γ_k small enough.

a) Implement the steepest descent method using the functions mentioned above, with your own strategy for selecting γ_k.

b) Using the solution (unnormalized cameras and 3D points for the inlier matches) that you obtained in Exercise 5 test how much you can decrease the objective value with your steepest descent implementation. Plot the RMS error vs. iteration number for 10 iterations and compute the final error.

Note that you might need a very small γ_k to converge! Plot the RMS error vs. iteration number for 10 iterations and compute the final error.

For the report: Submit the code, the plot and the final RMS value from (***b***).

Exercise 7. In the Levenberg-Marquardt method we instead choose the update step as

$$\delta v = -(J(v_k)^T J(v_k) + \lambda I)^{-1} J(v_k)^T \mathbf{r}(v_k) \tag{4}$$

(***a***) Implement the Levenberg-Marquardt method with some appropriate value for λ.

(***b***) Starting from the solution to Exercise 5 test how much you can decrease the objective value and compare to the steepest descent implementation. Plot the RMS error vs. iteration number for 10 iterations and compute the final error.

For the report: Submit the code, the plot and the final RMS value from (***b***).

```
Hints:

#Computes the LM update.
delta = np.linalg.solve(J.T @ J + lamb * np.eye(J.shape[1]), -J.T @ r)
```

---

5 Building a Structure-from-Motion System

Exercise 8. (OPTIONAL) We will now put all of the components together in our own Structure-from-Motion system! In the file `fman95sfm.py` you can find a skeleton incremental Structure-from-Motion which takes care of some of the bookkeeping necessary (keeping track of which 2D point is associated with which 3D point, etc.). The pipeline calls `fman95sfm_impl.py` which has several placeholder functions for you to implement:

- `select_initial_pair` – Select the initial image pair for bootstrapping
- `select_next_image` – Select the next image to register
- `ransac_essential_matrix` – RANSAC estimation of the essential matrix
- `ransac_camera_pose` – RANSAC estimation of camera pose from 2D-3D correspondences
- `triangulate_pair` – Triangulation of 3D points from two views
- `compute_reprojection_errors` – Compute reprojection errors for 3D points
- `bundle_adjustment` – Bundle adjustment optimization

Feel free to re-use your implementations from previous assignments. You can make any changes you want to `fman95sfm.py`. Try to make the SfM system as robust as possible.

In `A4_ex8_data.npz` you can find the K matrix, keypoints and matches for the twelve images `house0.jpg`, ..., `house11.jpg`. You can also run on your own images, but remember that you need to calibrate your camera (estimate the K matrix) in this case.

The bundle adjustment quickly becomes very slow as more 3D points and cameras are added. You can replace the LM update with the slightly faster code in the `Hints:` section below which use sparse matrices and iterative solvers to speed it up.

For the report: This exercise is optional, but if you want bonus for it submit code and plots of the resulting reconstruction. Also describe any design choices you made in your system.

```
Hints:

# To speed up the bundle adjustment you can use sparse matrices
import scipy.sparse
r, J = utils.linearize_reprojection_error(P, x, X)
J = scipy.sparse.csr_matrix(J, copy=False)
# This is equivalent to the standard LM update
delta = scipy.sparse.linalg.lsmr(J, -r, damp=np.sqrt(lam))[0]
```